

Module 2: Syntax Analysis

Context-Free Grammars, LL(1), SLR(1), CLR(1), LALR(1) & Parsing Conflicts

Module II: Syntax Analysis & Parsing Algorithms – Complete Syllabus Topics

- | | |
|--|---|
| 1. Role of Parser & Context-Free Grammars (CFG) Mathematical Formulations | 2. Parse Trees, Derivations (Leftmost & Rightmost) & Ambiguity Proofs |
| 3. Left Recursion Elimination (Immediate & Indirect Grammatical Algorithms) | 4. Left Factoring Algorithm for Common Prefixes |
| 5. FIRST and FOLLOW Sets Formal Inductive Formulations & Traces | 6. LL(1) Top-Down Predictive Parsing Table Construction & Stack Simulations |
| 7. Bottom-Up Shift-Reduce Parsing Principles & Handle Pruning Theorems | 8. LR(0) Canonical Collection of Items & SLR(1) Parsing Table Formulations |
| 9. Canonical LR (CLR(1)) Items with Lookaheads & LALR(1) State Merging | 10. Parsing Conflicts (Shift/Reduce, Reduce/Reduce) & Ambiguity Resolutions |
| 11. Syntax Error Detection & Recovery Strategies (Panic Mode & Phrase Level) | 12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems) |

Topic 1 & 2: Context-Free Grammars, Derivations & Ambiguity

A **Context-Free Grammar (CFG)** is formally defined as a 4-tuple $G = (V, T, P, S)$:

V : Finite set of Non-Terminal characters (syntactic variables).

T : Finite set of Terminal characters (token types returned by lexical analyzer).

P : Finite set of Production rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.

$S \in V$: The designated Start Symbol.

1. Derivations & Parse Trees:

Leftmost Derivation (LMD): At each derivation step, the leftmost non-terminal is replaced by the body of one of its productions ($\alpha \xRightarrow{lm} \beta$).

Rightmost Derivation (RMD / Canonical Derivation): At each step, the rightmost non-terminal is replaced ($\alpha \xRightarrow{rm} \beta$).

Bottom-up parsers reconstruct the reverse of a rightmost derivation (RMD^R).

Parse Tree: A graphical representation of a derivation where the root is start symbol S , interior nodes are non-terminals, and leaves are terminals or ϵ , read left-to-right to yield the input sentence.

Formal Definition of Grammar Ambiguity

A grammar G is **ambiguous** if there exists at least one string $w \in L(G)$ that has **two or more distinct parse trees** (or equivalently, two distinct leftmost derivations). Ambiguity makes deterministic parsing impossible unless resolved by disambiguating rules.

Classic Problem: Disambiguating the Dangling-Else Grammar

Ambiguous Grammar:

$$\text{stmt} \rightarrow \text{if expr then stmt} \mid \text{if expr then stmt else stmt} \mid \text{other}$$

For input `if E1 then if E2 then S1 else S2`, the `else` clause can be attached to the first or second `if`.

Unambiguous Disambiguated Grammar:

```
stmt      → matched_stmt | open_stmt
matched_stmt → if expr then matched_stmt else matched_stmt | other
open_stmt  → if expr then stmt
            | if expr then matched_stmt else open_stmt
```

Enforces that every `else` matches the closest preceding unmatched `then`.

Topic 3 & 4: Grammar Transformations (Left Recursion & Left Factoring)

1. Immediate Left Recursion Elimination:

A production of the form $A \rightarrow A\alpha \mid \beta$ causes top-down predictive parsers to loop infinitely. It is eliminated by introducing a new non-terminal A' :

$$A \rightarrow \beta A', \quad A' \rightarrow \alpha A' \mid \epsilon$$

2. General Left Recursion Elimination Algorithm (Handling Indirect Recursion):

Order non-terminals in an arbitrary sequence: $A_1, A_2, \dots, A_n$.

For $i = 1$ to n :

For $j = 1$ to $i - 1$: Replace each production of the form $A_i \rightarrow A_j \gamma$ by $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$, where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ are the current A_j rules.

Eliminate immediate left recursion among the A_i productions.

3. Left Factoring Algorithm (Removing Common Prefixes):

When two productions for A share a common prefix ($A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$), the parser cannot determine which rule to apply with 1 lookahead token. Left factoring delays the decision:

$$A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2$$

Topic 5: Formal Formulations of FIRST and FOLLOW Sets

Mathematical Definitions

- $\text{FIRST}(\alpha)$: Set of all terminals that appear as the first symbol in strings derived from α . If $\alpha \xrightarrow{*} \epsilon$, then $\epsilon \in \text{FIRST}(\alpha)$.
- $\text{FOLLOW}(A)$: Set of all terminals a that can appear immediately to the right of non-terminal A in some sentential form ($S \xrightarrow{*} \alpha A a \beta$). If A is the rightmost symbol in a derivation, the input end-marker $\$$ is in $\text{FOLLOW}(A)$.

Formal Rules for Computing FIRST:

If X is a terminal, $\text{FIRST}(X) = \{X\}$.

If $X \rightarrow \epsilon$ is a production, add ϵ to $\text{FIRST}(X)$.

If $X \rightarrow Y_1 Y_2 \dots Y_k$ is a production:

Add all non- ϵ symbols of $\text{FIRST}(Y_1)$ to $\text{FIRST}(X)$.

If $\epsilon \in \text{FIRST}(Y_1)$, add non- ϵ symbols of $\text{FIRST}(Y_2)$, continuing until some Y_i does not contain ϵ .
If all $Y_1, \dots, Y_k$ contain ϵ , add ϵ to $\text{FIRST}(X)$.

Formal Rules for Computing FOLLOW:

Place \$ in $\text{FOLLOW}(S)$, where S is the grammar start symbol.
If there is a production $A \rightarrow \alpha B \beta$, everything in $\text{FIRST}(\beta)$ except ϵ is in $\text{FOLLOW}(B)$.
If there is a production $A \rightarrow \alpha B$, or $A \rightarrow \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ , everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

 Step-by-Step Solved Problem: Arithmetic Grammar FIRST & FOLLOW

Transformed Arithmetic Grammar (G):

- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid \text{id}$

Non-Terminal	FIRST Set	FOLLOW Set
E	{(, id}	{), \$}
E'	{+, ϵ }	{), \$}
T	{(, id}	{+,), \$}
T'	{*, ϵ }	{+,), \$}
F	{(, id}	{*, +,), \$}

Topic 6: Top-Down LL(1) Predictive Parsing

An **LL(1) Parser** scans input from **Left-to-right**, produces a **Leftmost** derivation, using **1** token of lookahead without backtracking.

Formal Condition for an LL(1) Grammar

A context-free grammar G is $LL(1)$ if and only if for every pair of productions $A \rightarrow \alpha \mid \beta$:

- $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$.
- If $\epsilon \in \text{FIRST}(\alpha)$, then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$.

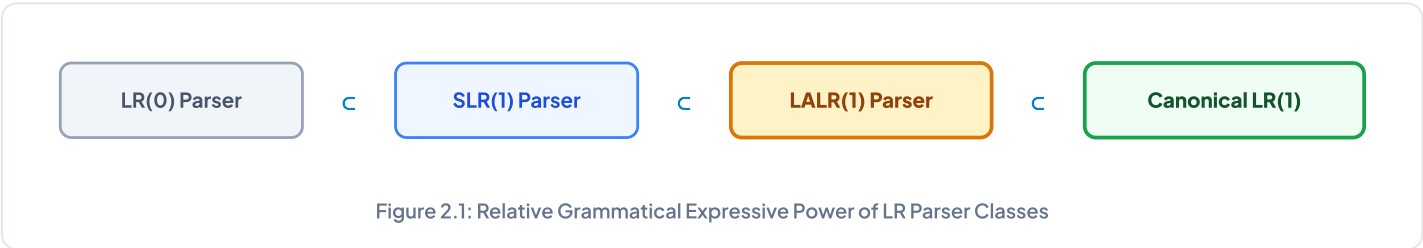
Complete LL(1) Parsing Table $M[A, a]$:

Non-Terminal	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Since no cell contains multiple production rules, the grammar is strictly $LL(1)$.

Topics 7 – 9: Bottom-Up LR Parsing Hierarchy

LR Parsers (Left-to-right scan, Rightmost derivation in reverse) are the most general deterministic shift-reduce parsing engines.



Parser Class	Item Representation	Reduce Action Decision Logic	State Complexity & Practical Use
1. LR(0)	$[A \rightarrow \alpha \cdot \beta]$	Reduce $A \rightarrow \alpha$ across all columns in row unconditionally.	Fewest states; very weak; fails on almost all programming grammars.
2. SLR(1)	$[A \rightarrow \alpha \cdot \beta]$	Reduce $A \rightarrow \alpha$ only on input terminal $a \in \text{FOLLOW}(A)$.	Same states as LR(0); resolves many simple shift/reduce conflicts.
3. CLR(1)	$[A \rightarrow \alpha \cdot \beta, a]$ (with explicit lookahead a)	Reduce $A \rightarrow \alpha$ only on exact lookahead terminal a .	Most powerful deterministic class; massive state explosion (thousands of states).
4. LALR(1)	Merged LR(1) items having identical LR(0) core items.	Reduce on merged lookahead terminal sets.	Same number of states as SLR(1); nearly CLR(1) parsing power; standard in Yacc/Bison.

Topic 10 & 11: Parsing Conflicts & Error Recovery

1. Bottom-Up Parsing Conflicts:

Shift/Reduce (S/R) Conflict: The parser state cannot decide whether to shift the next input symbol onto the stack or reduce the current top-of-stack handle by a production rule.

Reduce/Reduce (R/R) Conflict: The parser state recognizes two or more distinct completed handles simultaneously and cannot decide which production to reduce by (indicates severe grammatical ambiguity).

2. Error Recovery Modes:

Panic-Mode Recovery: The parser discards incoming tokens until a designated synchronizing token (e.g., `;`, `}`) is encountered, then pops stack symbols until parsing can safely resume. Simple and guaranteed to avoid infinite loops.

Phrase-Level Recovery: Replaces an erroneous prefix with a valid correction (e.g., replacing comma with semicolon).

Error Productions: Language designers augment the grammar with common anticipated developer errors to emit detailed diagnostic messages.



Top BIT Mesra Exam Questions & Answers (Module II)

Q1. Prove why an ambiguous grammar can never be LL(1) or LR(1). (8 Marks)

Proof: An ambiguous grammar generates at least two distinct parse trees for some valid string w .

1. In an **LL(1) parser**, this ambiguity manifests as multiple production entries in the same parsing table cell $M[A, a]$, violating the single-entry requirement of deterministic LL(1) tables.
2. In an **LR(1) parser**, the two distinct derivations create either a **Shift/Reduce conflict** (where the parser cannot decide whether to shift or reduce) or a **Reduce/Reduce conflict** (where the parser cannot decide which of two distinct productions to reduce by). Thus, ambiguous grammars can never be parsed deterministically by LL(1) or LR(1).

Q2. Explain the difference between SLR(1) and LALR(1) parsers. Can merging states introduce Shift/Reduce conflicts in LALR(1)? (10 Marks)

- **SLR(1):** Uses LR(0) items and makes reduction decisions based on the global $FOLLOW(A)$ set, which often includes symbols that cannot legally appear in that specific context.
- **LALR(1):** Merges CLR(1) states having identical LR(0) cores while unioning their lookahead sets. Reduction is guided by exact propagated lookaheads.
- **Theorem on Conflicts:** Merging LR(1) states with identical cores **CANNOT introduce new Shift/Reduce conflicts** (because shift decisions depend solely on the core item's symbol after the dot). However, merging **CAN introduce new Reduce/Reduce conflicts** if two distinct productions have overlapping lookaheads after merging.